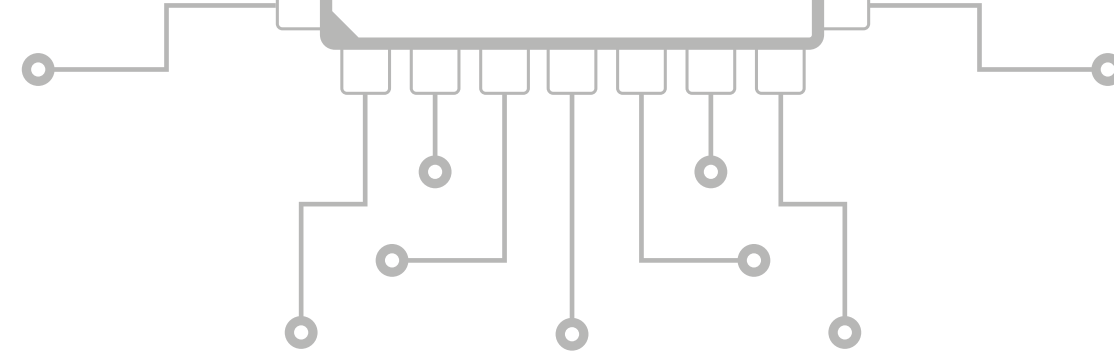
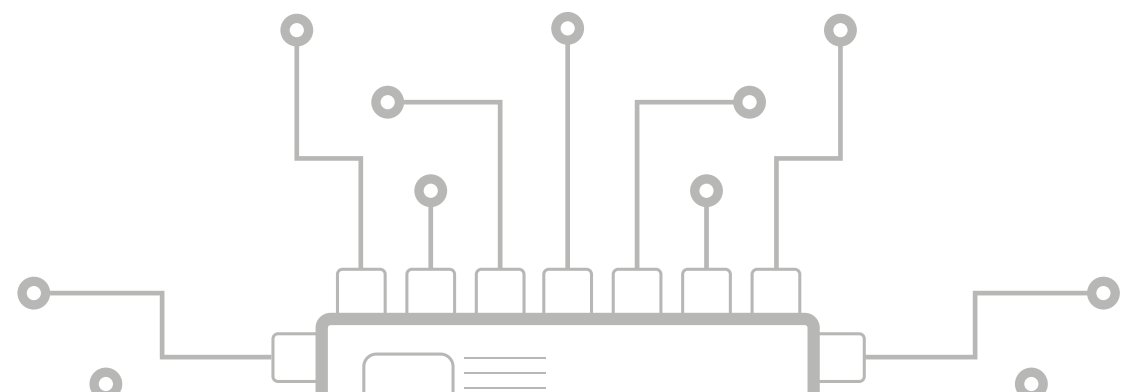




# Úvod do programování

Základní pojmy, programovací jazyky,  
způsoby programování

**Dominik Němeček**



# Programování

- Programování je **proces vytváření instrukcí, které počítač provede**. Tyto instrukce se zapisují ve formě kódu v konkrétním programovacím jazyce.
- Programování **umožňuje vývojářům vytvořit software**, který může vykonávat určité úkoly, od jednoduchých operací jako je sčítání čísel až po složité systémy jako jsou videohry, operační systémy nebo webové aplikace.



# Program

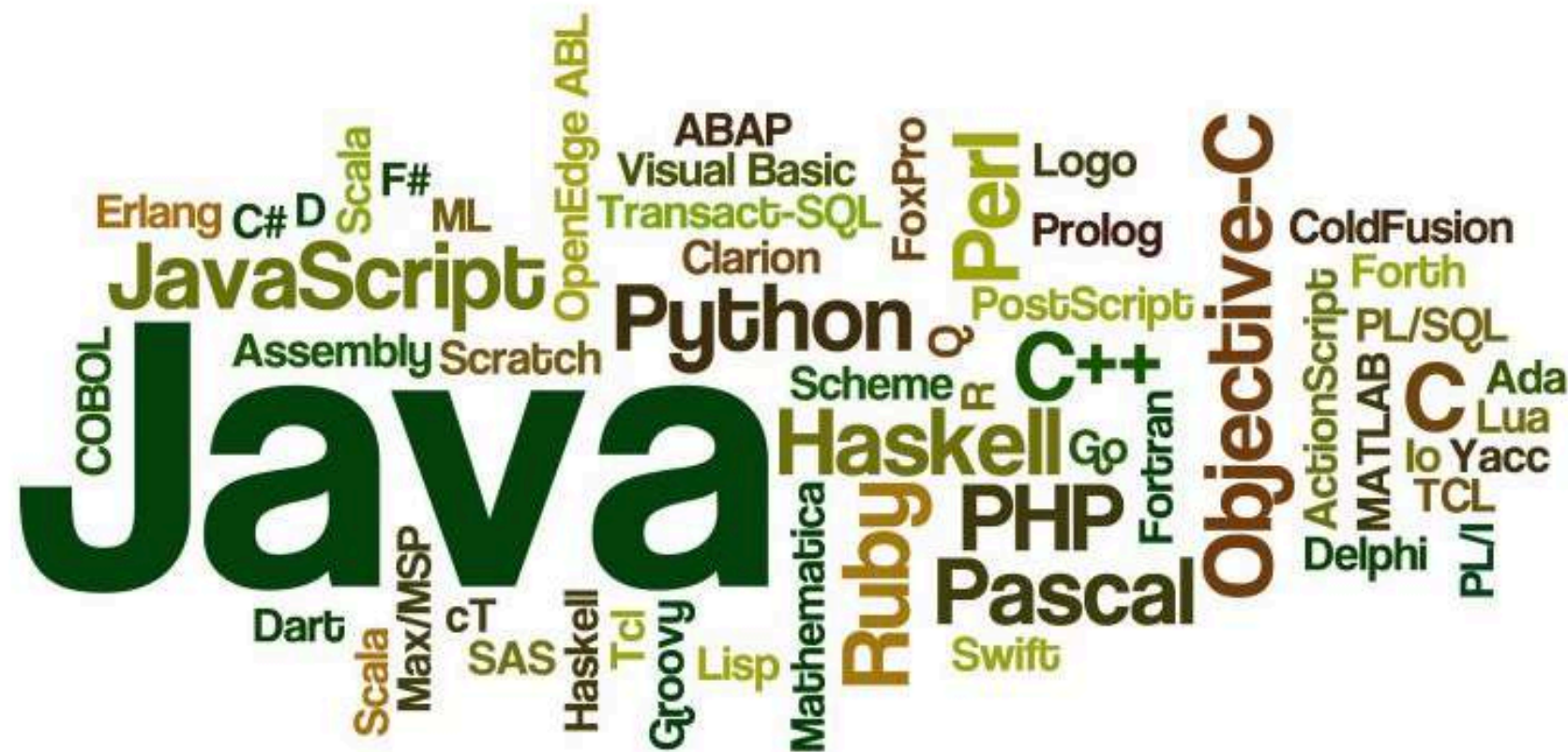
- Program je **sada instrukcí napsaná v určitém programovacím jazyce**, které počítač vykonává.
- Programy mohou být jednoduché, jako například kalkulačka, nebo složité, jako je software pro zpracování textu či aplikace pro správu databází.
- **Každý program má svou strukturu**, která zahrnuje proměnné, podmínky, cykly, funkce atd., aby mohl plnit svůj zamýšlený účel.



```
state={
  products: storeProducts
}
render() {
  return (
    <React.Fragment>
      <div className="py-5">
        <div className="container">
          <Title name="our" title="product">
            <div className="row">
              <ProductConsumer>
                {(value) => {
                  console.log(value)
                }}
              </ProductConsumer>
            </div>
          </div>
        </div>
      </React.Fragment>
    )
  }
}
```

# Programovací jazyk

- Programovací jazyk je **formální jazyk, který umožňuje lidem komunikovat s počítačem.**
- Tento jazyk se **skládá z pravidel (syntaxe) a slovní zásoby (klíčových slov a symbolů), které společně umožňují psát instrukce,** jež počítač může vykonávat.
- Existuje více programovacích jazyků, přičemž každý z má svou syntaxi a jiné využití.



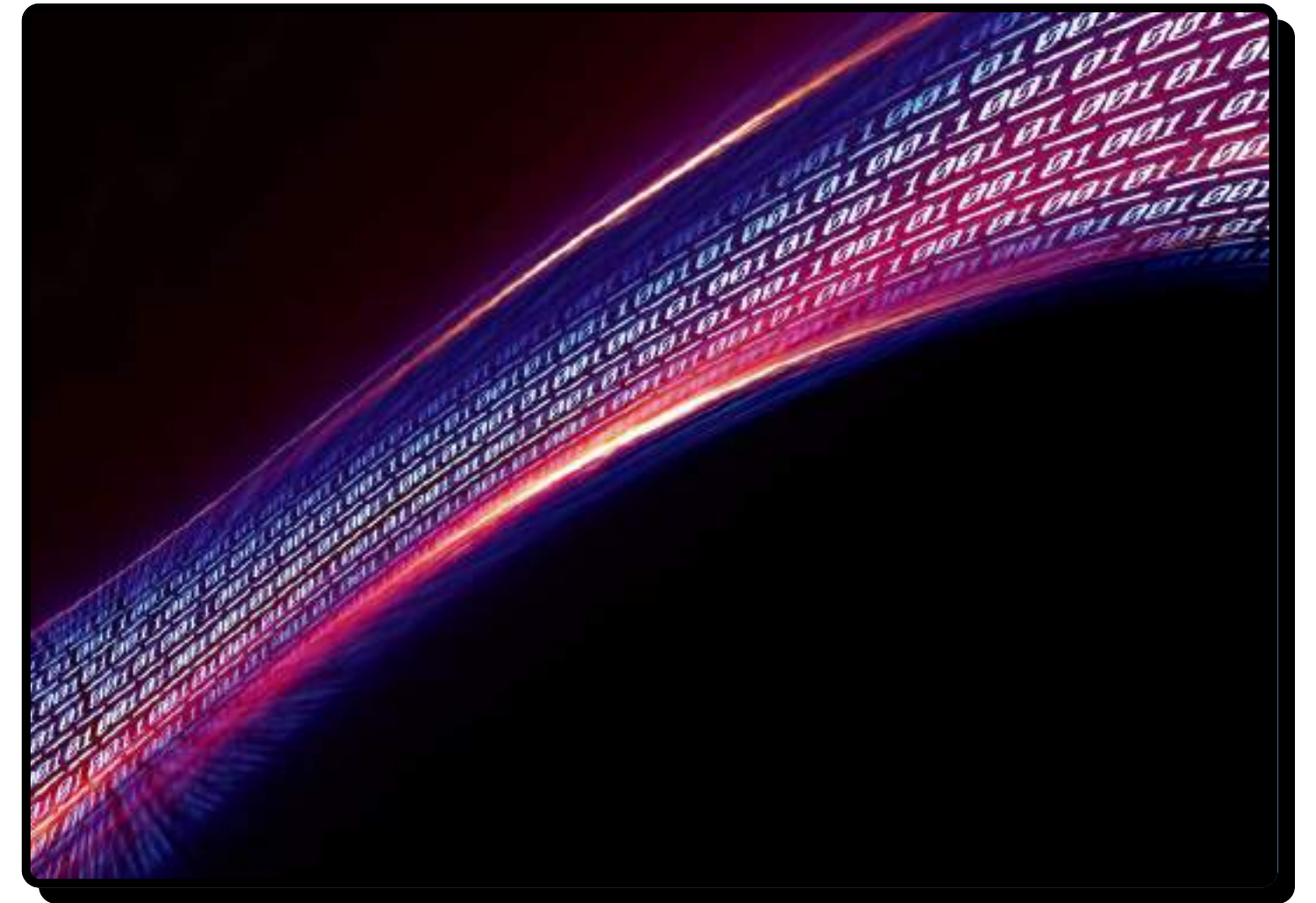
# Syntaxe

- Syntaxe v programování je **soubor pravidel pro správný zápis příkazů a struktur v daném jazyce**.
- **Určuje povolené kombinace symbolů a klíčových slov**, aby kód mohl být správně interpretován či zkompilován.
- **Porušení těchto pravidel vede k syntaktické chybě** (syntax error), kterou kompilátor nebo interpret nedokáže zpracovat.



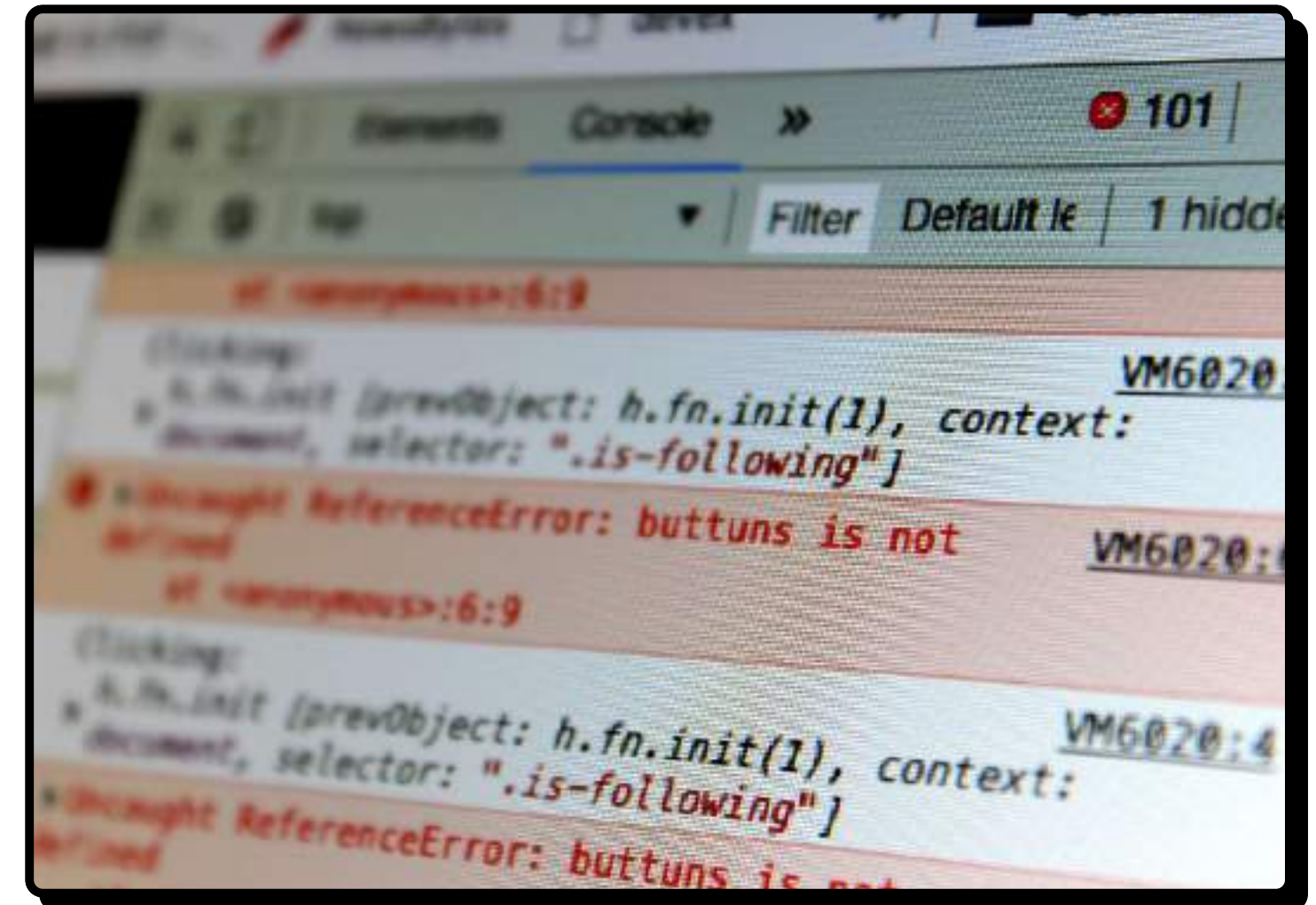
# Sémantika

- Sémantika v programování se **zabývá významem a chováním kódu při jeho spuštění.**
- Zatímco syntaxe určuje, jak má být kód napsán, **sémantika se zaměřuje na to, co příkazy znamenají a jak ovlivňují stav programu.**
- Jinými slovy, sémantika odpovídá na otázku: **"Co se stane, když tento kód spustím?"**



# Debugging

- Debugging (ladění) je proces v programování, při kterém se **hledají a opravují chyby v kódu, které způsobují nesprávné chování programu.**
- **Cílem debuggování je identifikovat problémy,** porozumět jim a najít řešení, aby program fungoval podle očekávání.
- Pojem vychází **z dob HW programování (odstraňování broků).**



# Zdrojový kód

- Jedná se o **textový soubor nebo kolekce textových instrukcí napsaných v určitém programovacím jazyce.**
- Tento kód **slouží jako lidsky čitelná forma programu**, který programátoři, upravují a udržují.
- Jedná se tedy o **text, který představuje "recept" na to, jak by měl program fungovat**, a slouží jako základní materiál, který se převádí do spustitelné podoby.

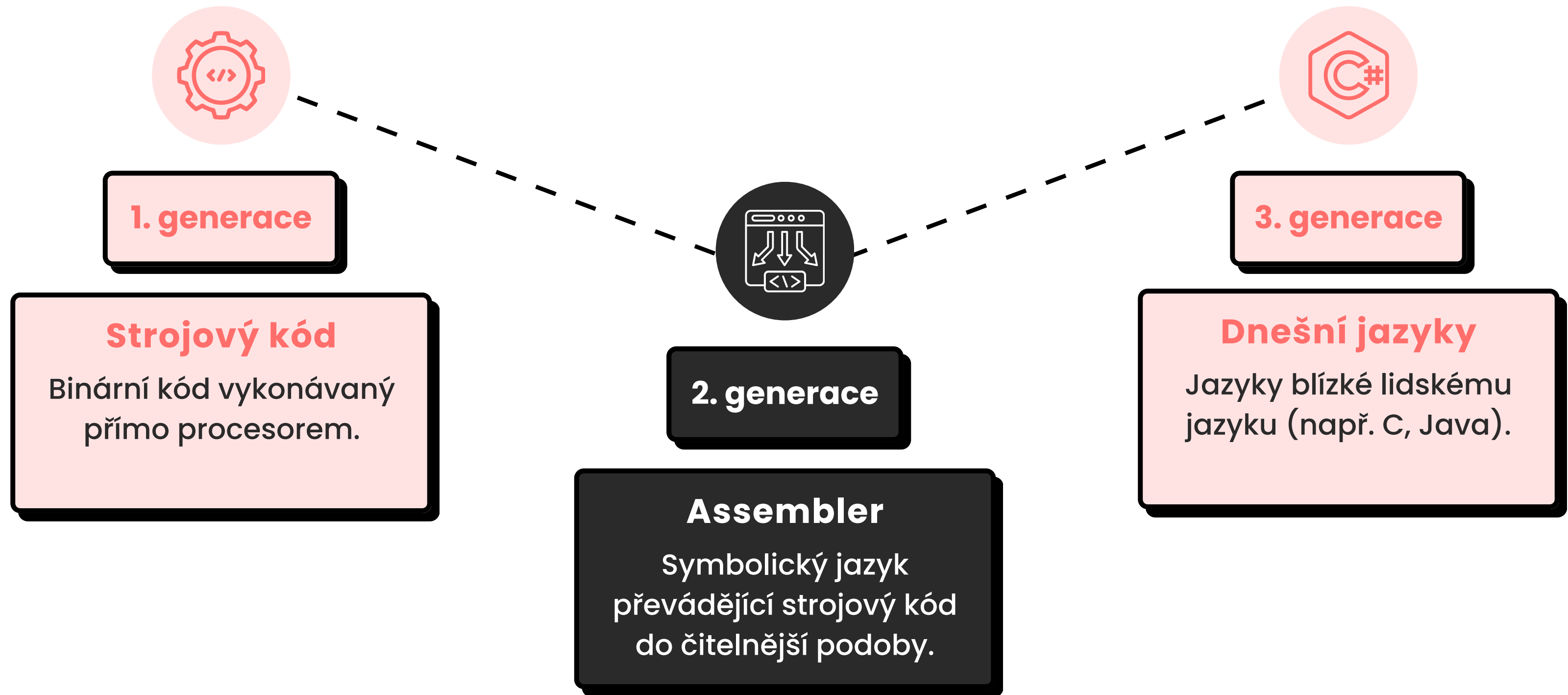
```
<div class="form-box login-box active">
  <form class="form-signin">
    <input type="text" name="username" value="{{username}}"/>
    Email address
    <input type="password" name="password" value="{{password}}"/>
    placeholder="Password"
    <input type="checkbox" id="login-remember"/>
    <label for="login-remember">Keep me logged in
    <button class="btn btn-tall btn-wide">Sign in
  </form>
  {{#error}}
  <div class="error-message">{{error}}</div>
</div>
```

# Strojový kód

- Strojový kód je **nejnižší úroveň binárních instrukcí (0 a 1), které počítač přímo vykonává**, a je specifický pro daný procesor nebo počítačovou architekturu.
- Tento kód **představuje přímé pokyny pro procesor, které říkají, jaké operace má provést**, například přesun dat, aritmetické operace, nebo řízení toku programu.



# Vývoj programovacích jazyků



# 1. generace – strojový kód

- Strojový kód je nejzákladnější formou programování. **Je to jazyk, který je přímo interpretován procesorem a je zapsán v binární formě (0 a 1).**
- Každá instrukce přímo odpovídá operaci, kterou procesor provádí.

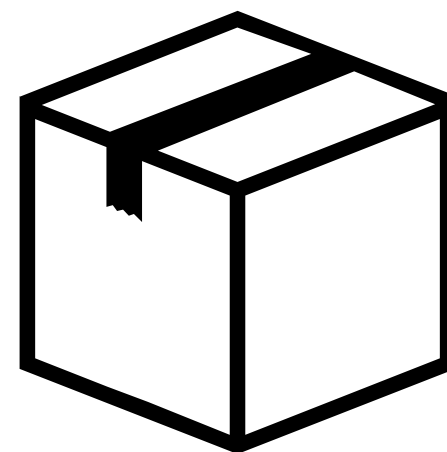
```
2104
1105
3106
7001
0053
0000
```

## Výhody

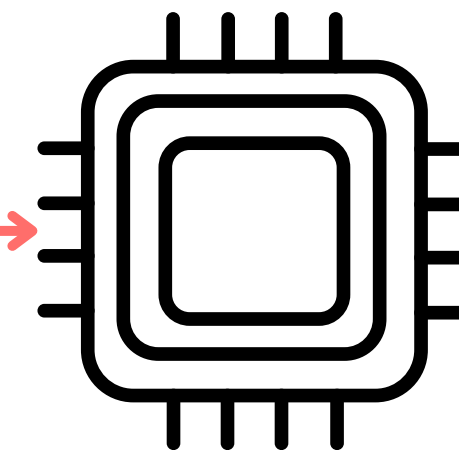
- ✓ Maximální výkon
- ✓ Kontrola nad HW
- ✓ Nízká spotřeba paměti

## Nevýhody

- ✗ Náročnost psaní
- ✗ Údržba a ladění
- ✗ Nekompabilita



**Strojový kód**



**Procesor**



## 2. generace – Assembler

- Assembler je jazyk nízké úrovně, který využívá **symbolické instrukce namísto binárního kódu**.
- **Každá instrukce assembleru odpovídá jedné instrukci strojového kódu**, ale používá čitelnější symboly (např. MOV pro přesun dat, ADD pro sčítání).

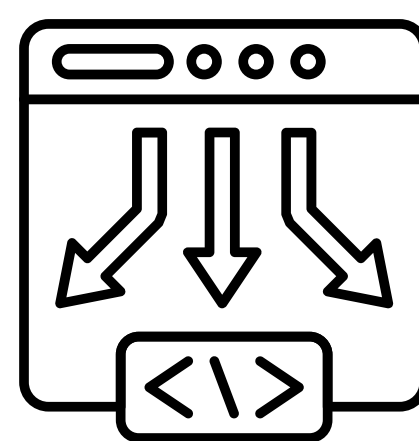
```
ORG 100  
LDA A  
ADD B  
STA C  
HLT  
DEC 83
```

### Výhody

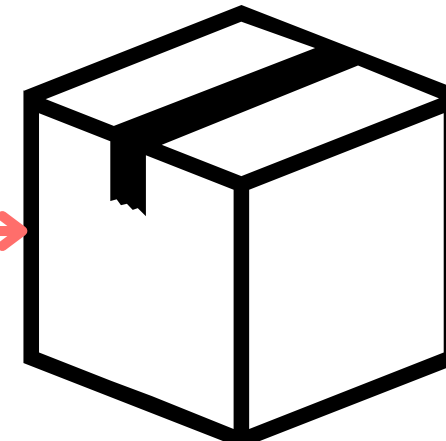
- ✓ Vysoký výkon
- ✓ Kontrola nad HW
- ✓ Správa paměti

### Nevýhody

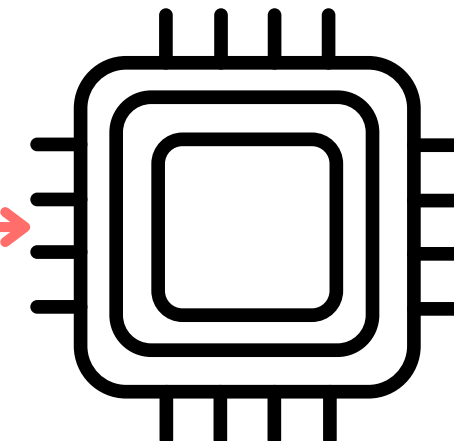
- ✗ Složitost
- ✗ Údržba a ladění
- ✗ Nekompabilita



Assembler



Strojový kód



Procesor

# 3. generace – dnešní jazyky

- Třetí generace programovacích jazyků **zlepšila abstrakci, což umožňuje psát kód blíže lidskému jazyku a logice.**
- Jazyky jako Fortran, COBOL, C, Pascal, Java a Python se **zaměřují na řešení problémů** místo na specifik hardware.

```
#include <stdio.h>
int x = 8;

int main()
{
    printf("%d", x);
}
```

## Výhody

- ✓ Jednoduché psaní
- ✓ Kompatibilita
- ✓ Lehčí údržba

## Nevýhody

- ✗ Spotřeba paměti
- ✗ Nepřímý přístup k HW
- ✗ Nižší výkon

(Výhody a nevýhody porovnáváme napříč všemi generacemi, neznámá to tedy, že dnešní jazyky mají velkou spotřebu paměti, znamená to, že mají větší spotřebu oproti 1. a 2. generaci.)

(Ukázka kódu jednoho z dnešních používaných jazyků, konkrétně se jedná o jazyk C.)

| Rozdělení dnešních jazyků<br>z hlediska překladu |                                                                                       |
|--------------------------------------------------|---------------------------------------------------------------------------------------|
| 1) Kompilované                                   |  |
| 2) Interpretované                                |                                                                                       |
| 3) S virtuálním strojem                          |                                                                                       |

# 3. generace – kompilované jazyky

- Kompilované jazyky jsou programovací jazyky, které se překládají do strojového kódu pomocí kompilátoru.
- Tento překlad **probíhá před vykonáním programu**, což znamená, že výsledný strojový kód je připraven k okamžitému spuštění na konkrétní platformě.

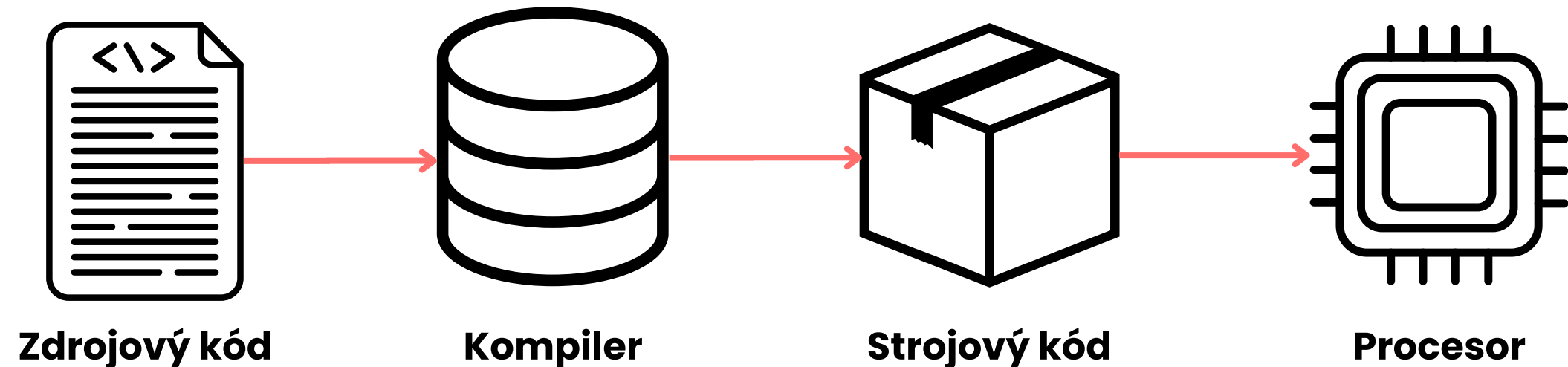
(Výhody a nevýhody porovnáváme napříč 3. generací jazyků.)

## Výhody

- ✓ Vyšší výkon
- ✓ Bezpečnost
- ✓ Snadná kontrola

## Nevýhody

- ✗ Závislost na platformě
- ✗ Nemožnost editace
- ✗ Správa paměti



# 3. generace – interpretované jazyky

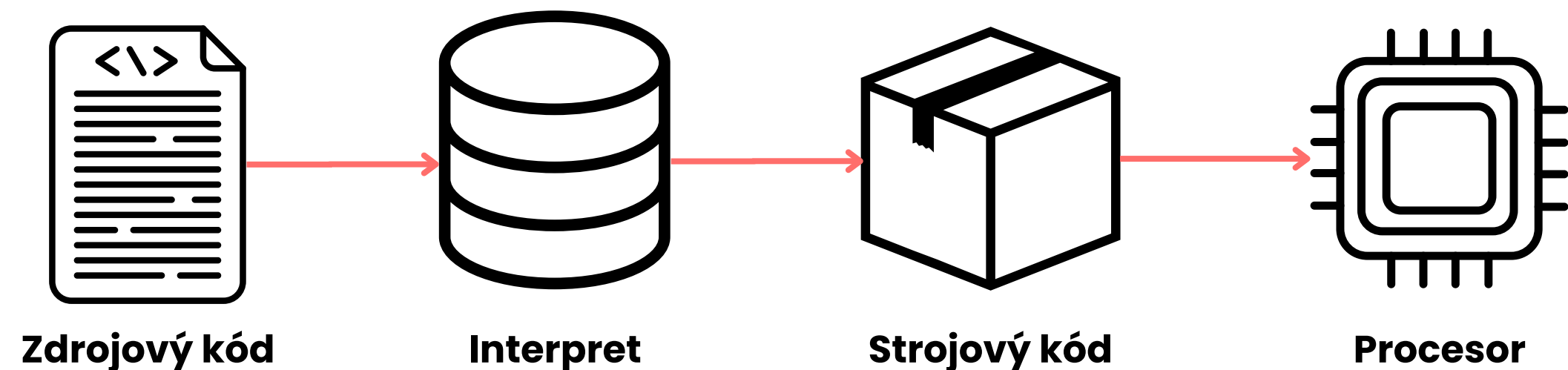
- Interpretované jazyky jsou programovací jazyky, jejichž kód je vykonáván přímo interpretem, který **provádí instrukce v reálném čase**.
- **Interpretace probíhá během běhu programu**, což může znamenat pomalejší výkon ve srovnání s kompilovanými jazyky.

## Výhody

- ✓ Přenositelnost
- ✓ Jednodušší vývoj
- ✓ Jednoduchá editace

## Nevýhody

- ✗ Nižší rychlost
- ✗ Hledání chyb
- ✗ Zranitelnost



# 3. generace – jazyky s virtuálním strojem

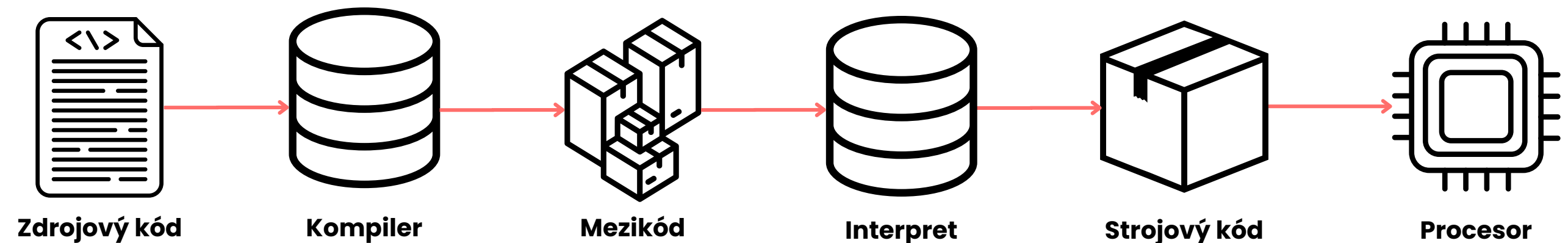
- Jazyky s virtuálním strojem (VM) používají prostředí, které **simuluje HW a poskytuje abstraktní vrstvy mezi kódem a skutečným hardwarem**.
- Kód napsaný v těchto jazycích **je obvykle překládán do bytecode (mezikód)**, který je následně vykonáván virtuálním strojem (kombinace kompilátoru a interpreta).

## Výhody

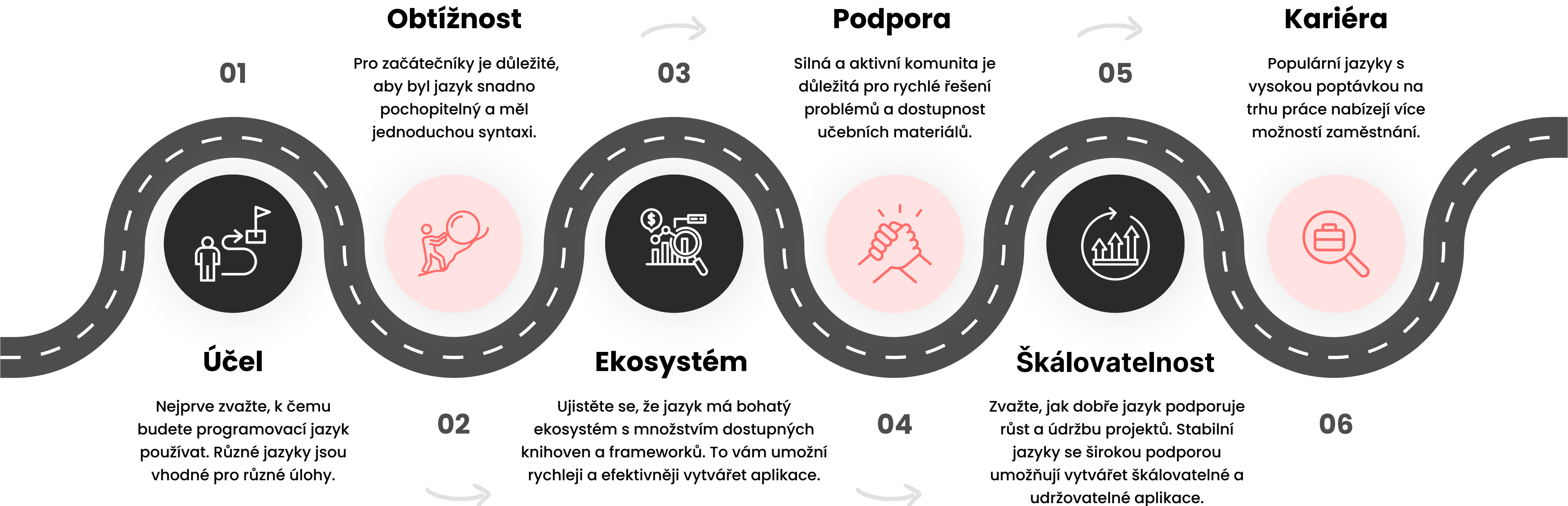
- ✓ Přenositelnost
- ✓ Bezpečnost
- ✓ Správa paměti

## Nevýhody

- ✗ Nižší rychlost (oproti kompilovanému)



# Volba programovacího jazyka



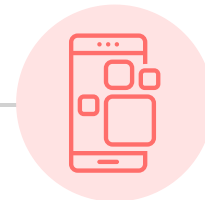
# Programovací jazyky

## z hlediska uplatnění

**WEBOVÝ VÝVOJ**



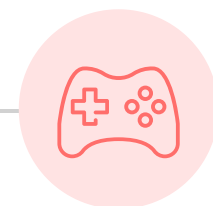
**MOBILNÍ APLIKACE**



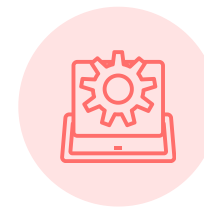
**DATOVÁ ANALÝZA**



**VIDEOHRY**



**SYSTÉMOVÉ APLIKACE & HW**



**DESKTOPOVÉ APLIKACE**



# Webový vývoj

## HTML/CSS



Základní stavební kameny pro strukturu a styl webových stránek.

## JavaScript



Skriptovací jazyk pro dynamické a interaktivní prvky na webu (animace, formuláře, aktualizace obsahu bez načítání stránky).

## Python



Používá se k webovému vývoji s frameworky jako Django nebo Flask, které umožňují rychlé a efektivní vytváření webových aplikací.

## PHP

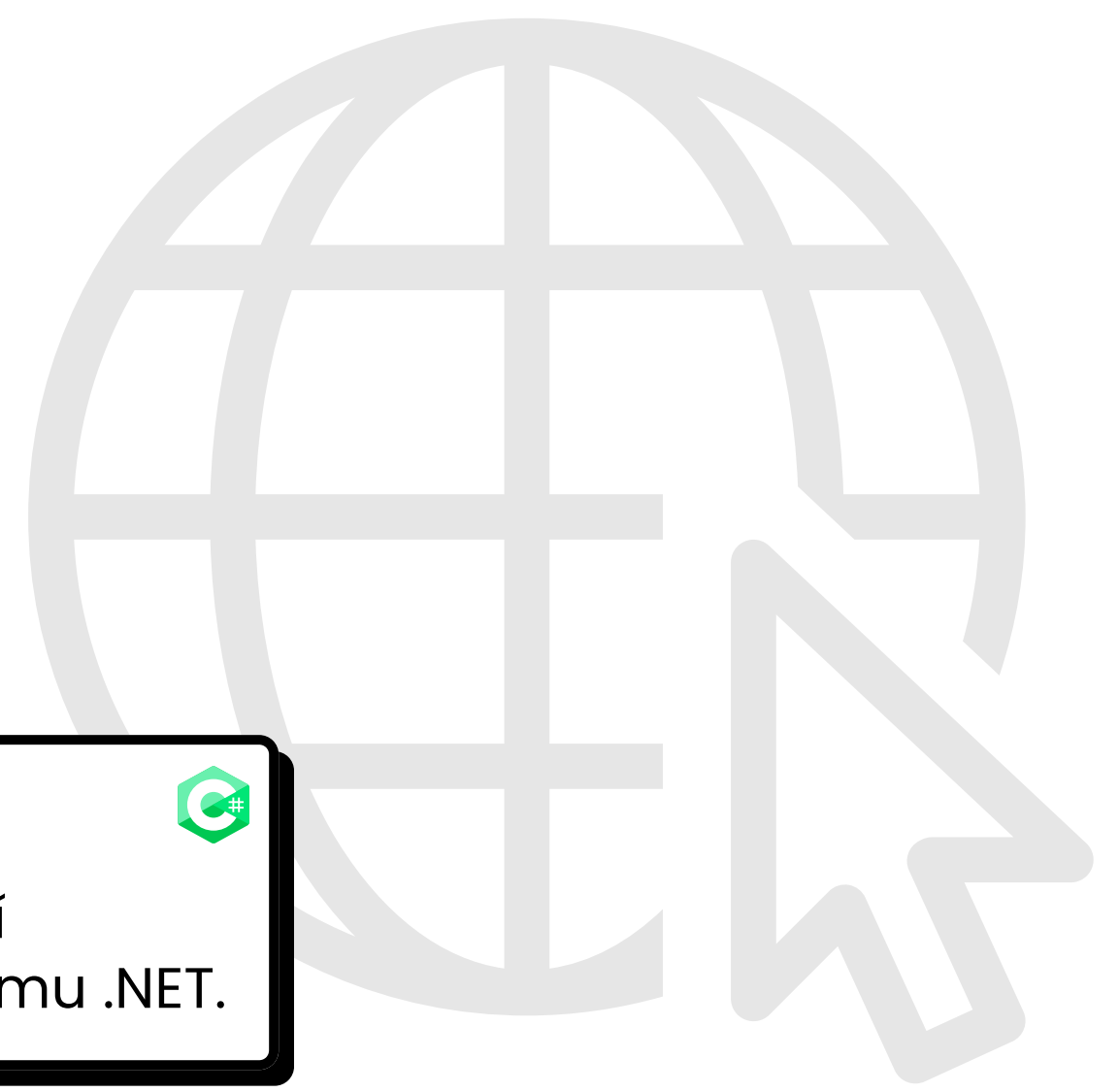


Serverový skriptovací jazyk, často používaný pro vytváření dynamických webových stránek a aplikací (generování HTML na straně serveru).

## C#



C# je moderní, objektově orientovaný programovací jazyk vyvinutý společností Microsoft, který se často používá pro vývoj webových aplikací v rámci ekosystému .NET.



# Mobilní aplikace

## Java



Java je tradiční a stále velmi populární jazyk pro vývoj aplikací na platformu Android.

## Swift



Swift je programovací jazyk vytvořený společností Apple pro vývoj aplikací na platformy iOS, macOS, watchOS a tvOS.

## Python



Python není běžně používán pro vývoj nativních mobilních aplikací, ale díky frameworkům jako Kivy nebo BeeWare je možné vytvářet mobilní aplikace, které běží na různých platformách.

## Kotlin

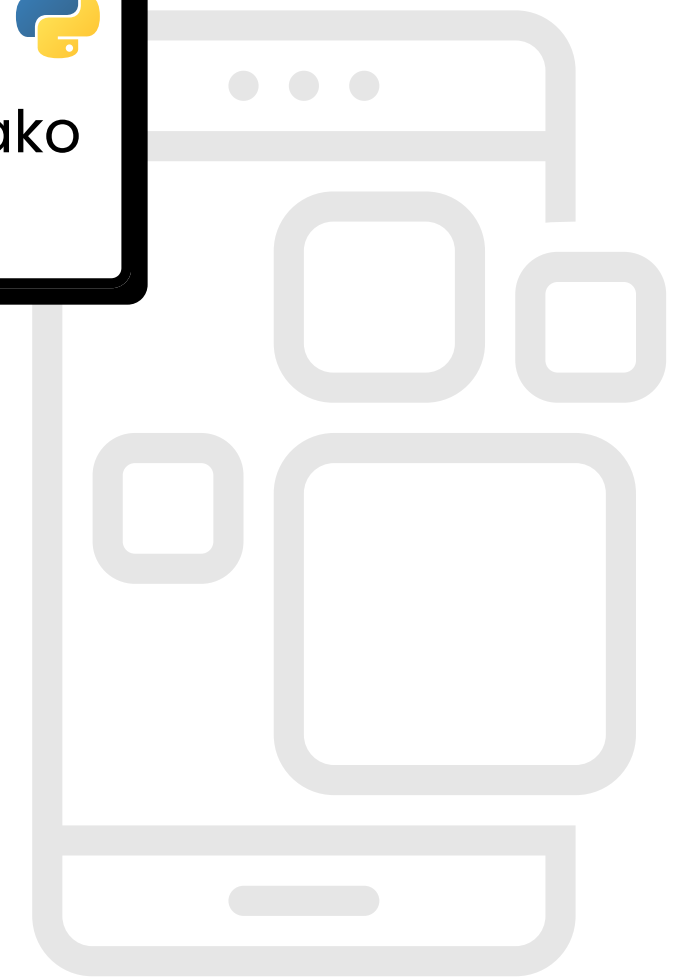


Kotlin je moderní programovací jazyk, který je oficiálně podporován společností Google pro vývoj Android aplikací.

## C#



C# je jazyk používaný ve frameworku Xamarin, který umožňuje vývoj aplikací pro Android, iOS a Windows pomocí jednoho sdíleného kódu.



# Datová analýza

## R



R je statistický programovací jazyk navržený speciálně pro datovou analýzu a vizualizaci.

## Python



Python je jedním z nejoblíbenějších jazyků pro datovou analýzu díky své jednoduché syntaxi a rozsáhlé sbírce knihoven, jako jsou Pandas (pro manipulaci s daty), NumPy (pro numerické výpočty) a Matplotlib nebo Seaborn (pro vizualizaci dat).

## SQL



SQL (Structured Query Language) je jazyk pro práci s databázemi, který umožňuje efektivně dotazovat, manipulovat a analyzovat strukturovaná data.

## MATLAB



MATLAB je výpočetní prostředí a programovací jazyk často používaný pro numerické analýzy, simulace a zpracování signálu.



# Videohry

## C++



C++ je jedním z nejpoužívanějších jazyků ve vývoji her díky své vysoké výkonnosti a kontrole nad hardwarem.

## Java



Java se používá pro vývoj her na různé platformy, včetně Androidu. Je oblíbená pro tvorbu mobilních her a menších projektů, ačkoli není tak rychlá jako C++ nebo C#.

## Python

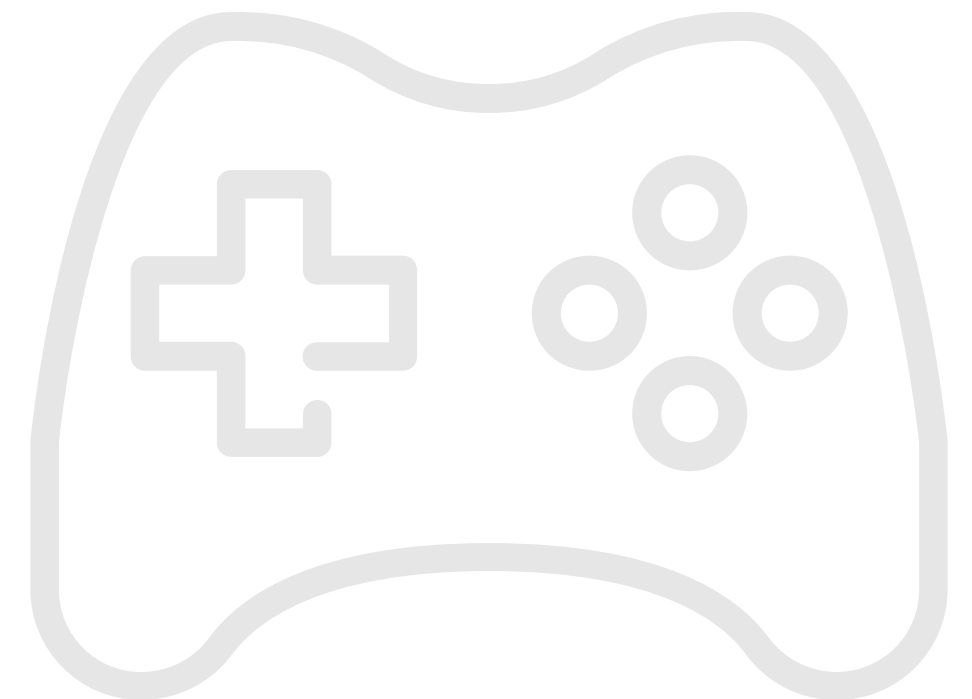


Python se používá hlavně pro vývoj jednoduchých her a prototypování díky své jednoduché syntaxi a knihovně Pygame, která poskytuje nástroje pro tvorbu 2D her.

## C#



C# je populární jazyk pro vývoj her, zejména díky široce používanému engineu Unity pro tvorbu 2D i 3D her.



# Systemové aplikace & HW

## C++



C++ rozšiřuje C o objektově orientované programování a je využíván pro vývoj systémových aplikací, herních enginů a dalších výkonných softwarových komponent.

## C



C je jedním z nejstarších a nejpoužívanějších jazyků pro tvorbu systémového softwaru, jako jsou operační systémy, ovladače a firmware.

## Rust



Rust je moderní programovací jazyk zaměřený na bezpečnost paměti a vysoký výkon. Díky těmto vlastnostem je ideální pro vývoj systémových aplikací a software pro vestavěné systémy, kde je důležitá spolehlivost a nízká režie.



# Desktopové aplikace

## C++



C++ je silný jazyk, který se využívá pro vývoj výkonných a efektivních desktopových aplikací. S knihovnamí jako Qt nebo wxWidgets lze vytvářet aplikace s moderním uživatelským rozhraním pro různé platformy.

## Java



Java je multiplatformní jazyk, který se často používá pro vývoj desktopových aplikací díky knihovnám jako Swing a JavaFX.

## C#



C# je populární jazyk pro vývoj desktopových aplikací na platformě Windows, zejména díky frameworku .NET a Windows Presentation Foundation (WPF).

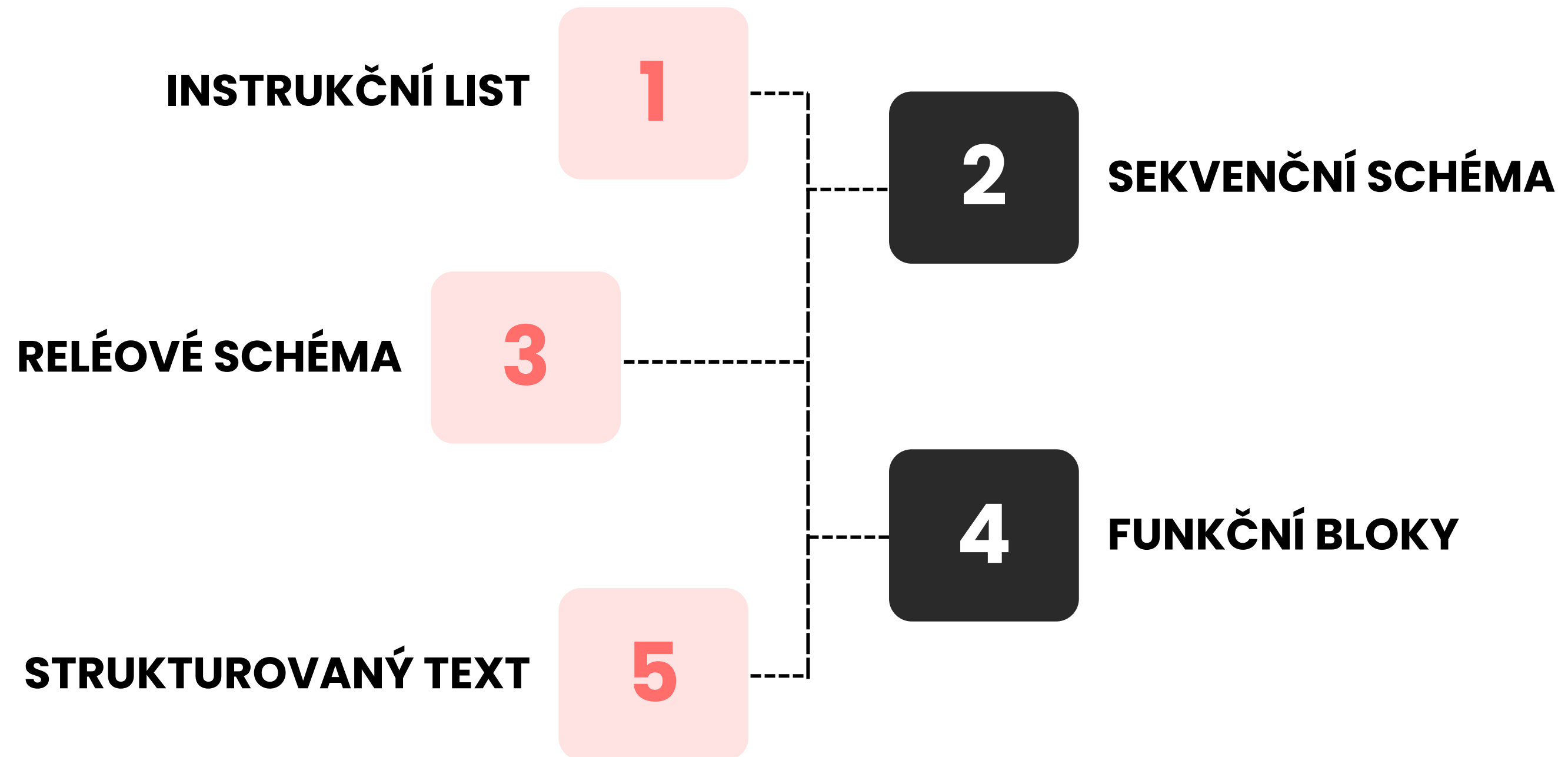
## Python



Python je často používán pro tvorbu desktopových aplikací díky své jednoduché syntaxi a knihovnám jako Tkinter, PyQt, nebo Kivy.



# Způsoby programování



# 1. Instrukční list

- Instrukční list je **seznam nebo sada instrukcí, které procesor vykonává, aby provedl určitou úlohu.**
- Každá instrukce typicky specifikuje operaci (např. přesun dat, aritmetický výpočet, logická operace) a operandy (např. registry, paměťové adresy).

## Výhody

- ✓ Přístup k HW
- ✓ Efektivita
- ✓ Přesnost

## Nevýhody

- ✗ Náročnost
- ✗ Čitelnost
- ✗ Podpora

|     |      |                |                                                                              |
|-----|------|----------------|------------------------------------------------------------------------------|
| 1.1 | LD   | SM 0.1         | On for One Scan                                                              |
| 1.2 | MOVD | # 4000 , VD200 | Put 4000 in address                                                          |
| 1.3 | LD   | SM 0.1         | VD200                                                                        |
| 1.4 | MOVW | # 41 , VW10    | Put 41 in address VW10                                                       |
| 1.5 | LD   | SM 0.1         |                                                                              |
| 1.6 | DIV  | VW10 , VD200   | DIV Value in Address<br>VD200 on the Value VW10<br>put result in the address |
| 1.7 | MEND |                | VW200                                                                        |

## 2. Sekvenční schéma

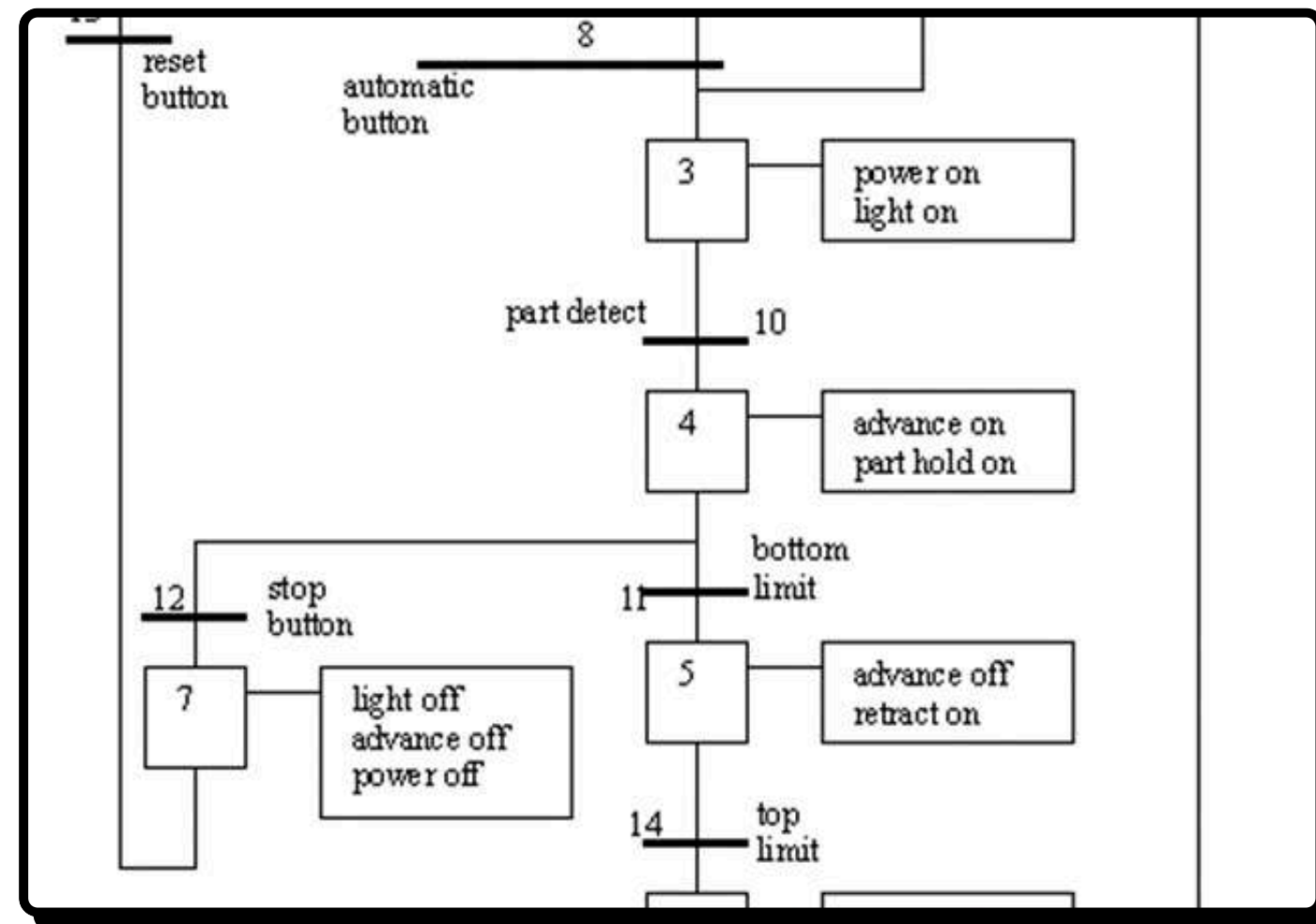
- Sekvenční schéma je **vizuální reprezentace, která zobrazuje postupné kroky nebo stavy v procesu řízení.**
- Každý krok je spojen s určitou akcí nebo operací a je aktivován na základě specifických podmínek, jako jsou vstupy od senzorů nebo dosažení určitého časového intervalu.

### Výhody

- ✓ Přehlednost
- ✓ Jednoduché ladění
- ✓ Modularita

### Nevýhody

- ✗ Omezená flexibilita
- ✗ Jednoduché projekty
- ✗ Pomalejší



# 3. Reléové schéma

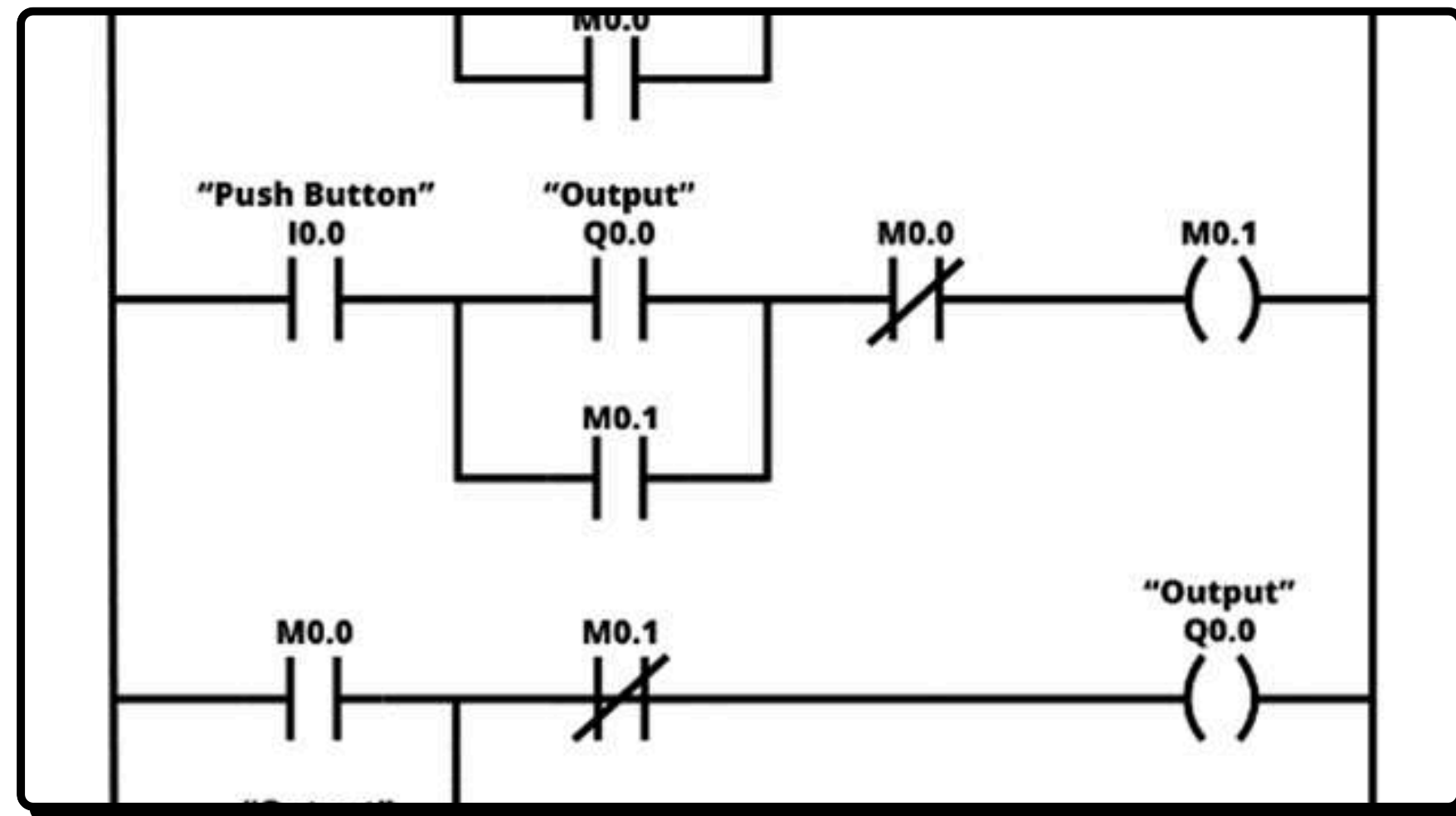
- Reléové schéma **představuje logiku řízení pomocí symbolů relé, kontaktů a spojů**, které napodobují fungování skutečných elektromechanických reléových obvodů.
- Tento grafický přístup umožňuje vizuálně navrhnout a implementovat řídicí logiku.

## Výhody

- ✓ Široká podpora
- ✓ Snadná diagnostika
- ✓ Jednoduchost

## Nevýhody

- ✗ Omezená expresivita
- ✗ Duplikace kódu
- ✗ Jednodušší projekty



# 4. Funkční bloky

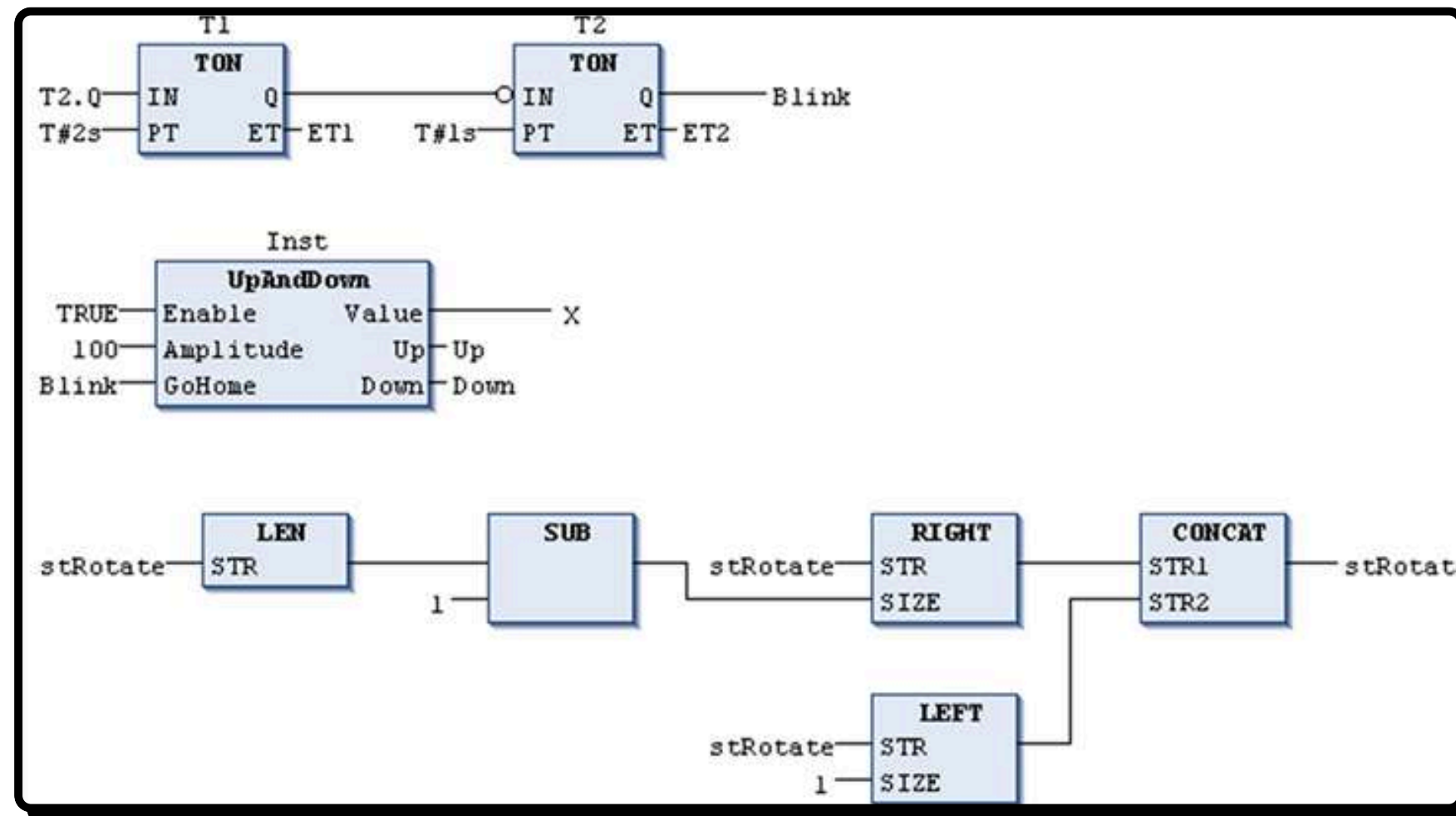
- Funkční bloky jsou jedním z mnoha způsobů programování v automatizačních systémech, **zejména v programovatelných logických automatech (PLC)**.
- Kombinuje vstupy, výstupy a interní logiku do jednoho celku, který provádí specifické úkoly, jako je řízení procesů nebo zpracování dat.

## Výhody

- ✓ Opakovaně použitelné
- ✓ Přehlednost
- ✓ Modularita

## Nevýhody

- ✗ Závislost na platformě
- ✗ Duplikace kódu
- ✗ Jednodušší projekty



# 5. Strukturovaný text

- Strukturovaný text je **textově orientované programování, které se podobá tradičním programovacím jazykům**, jako jsou Pascal, C nebo BASIC.
- Používá se k psaní složitějších algoritmů a logických operací v rámci automatizačních systémů a různých druhů SW.

## Výhody

- ✓ Přehlednost
- ✓ Podpora
- ✓ Modularita

## Nevýhody

- ✗ Složitost

```
9      double ...  
10     bool again = true;  
11  
12     while (again) {  
13         iN = -1;  
14         again = false;  
15         getline(cin, sInput);  
16         system("cls");  
17         stringstream(sInput) >> dblTemp;  
18         iLength = sInput.length();  
19         if (iLength < 4) {  
20             again = true;  
21             continue;  
22         } else if (sInput[iLength - 3] != '.') {  
23             again = true;  
24             continue;  
25         }  
26         while (++iN < iLength) {  
27             if (!isdigit(sInput[iN])) {  
28                 again = true;  
29                 continue;  
30             }  
31             if (iN % 3 == 0) {  
32                 cout << " " << endl;  
33                 continue;  
34             }  
35             cout << sInput[iN] << " " << endl;  
36         }  
37     }
```